

CONTROL AND OBSERVABILITY

Todo Lists

Track and display todos using the Claude Agent SDK for organized task management

Todo tracking provides a structured way to manage tasks and display progress to users. The Claude Agent SDK includes built-in todo functionality that helps organize complex workflows and keep users informed about task progression.

❗ As of TypeScript Agent SDK 0.3.142 and Claude Code v2.1.142, sessions use the structured Task tools `TaskCreate`, `TaskUpdate`, `TaskGet`, and `TaskList` instead of `TodoWrite`. See [Migrate to Task tools](#) for how monitoring code changes. The examples on this page set `CLAUDE_CODE_ENABLE_TASKS=0` to keep showing `TodoWrite` for sessions that have not migrated yet.

Todo Lifecycle

Todos follow a predictable lifecycle:

1. **Created** as `pending` when tasks are identified
2. **Activated** to `in_progress` when work begins
3. **Completed** when the task finishes successfully
4. **Removed** when all tasks in a group are completed

When Todos Are Used

The SDK automatically creates todos for:

- **Complex multi-step tasks** requiring 3 or more distinct actions
- **User-provided task lists** when multiple items are mentioned
- **Non-trivial operations** that benefit from progress tracking
- **Explicit requests** when users ask for todo organization

Examples

Monitoring Todo Changes

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Optimize my React app performance and track progress with todos",
  // Re-enable TodoWrite, which this example monitors. Without it, the SDK
  // uses
  // Task tools instead and these tool_use blocks never appear.
  options: { maxTurns: 15, env: { ...process.env, CLAUDE_CODE_ENABLE_TASKS:
"0" } }
})) {
  // Todo updates are reflected in the message stream
  if (message.type === "assistant") {
    for (const block of message.message.content) {
      if (block.type === "tool_use" && block.name === "TodoWrite") {
        const todos = block.input.todos;

        console.log("Todo Status Update:");
        todos.forEach((todo, index) => {
          const status =
            todo.status === "completed" ? "✅" : todo.status === "in_progress"
? "🔧" : "❌";
          console.log(`${index + 1}. ${status} ${todo.content}`);
        });
      }
    }
  }
}
```

Real-time Progress Display

```
import { query } from "@anthropic-ai/claude-agent-sdk";

class TodoTracker {
  private todos: any[] = [];

  displayProgress() {
    if (this.todos.length === 0) return;

    const completed = this.todos.filter((t) => t.status ===
"completed").length;
    const inProgress = this.todos.filter((t) => t.status ===
"in_progress").length;
    const total = this.todos.length;

    console.log(`\nProgress: ${completed}/${total} completed`);
    console.log(`Currently working on: ${inProgress} task(s)\n`);

    this.todos.forEach((todo, index) => {
      const icon =
        todo.status === "completed" ? "✅" : todo.status === "in_progress" ?
"🔧" : "❌";
      const text = todo.status === "in_progress" ? todo.activeForm :
todo.content;
      console.log(`${index + 1}. ${icon} ${text}`);
    });
  }

  async trackQuery(prompt: string) {
    for await (const message of query({
      prompt,
      // Re-enable TodoWrite, which this tracker watches for.
      options: { maxTurns: 20, env: { ...process.env,
        CLAUDE_CODE_ENABLE_TASKS: "0" } }
    })) {
      if (message.type === "assistant") {
        for (const block of message.message.content) {
          if (block.type === "tool_use" && block.name === "TodoWrite") {
            this.todos = block.input.todos;
            this.displayProgress();
          }
        }
      }
    }
  }
}
```

```

    }
  }
}

// Usage
const tracker = new TodoTracker();
await tracker.trackQuery("Build a complete authentication system with todos");

```

Migrate to Task tools

The Task tools split the single `TodoWrite` call into `TaskCreate` for each new item and `TaskUpdate` for each status change, with `TaskList` and `TaskGet` available for the model to read back the current list. Your monitoring code still inspects `tool_use` blocks in the assistant stream, but maintains a map keyed by task ID instead of replacing the whole list on every call. The Task tools are the default as of TypeScript Agent SDK 0.3.142 and Claude Code v2.1.142, so no `options.env` change is needed.

With <code>TodoWrite</code>	With Task tools
One tool call rewrites the full <code>todos</code> array	<code>TaskCreate</code> adds one item, <code>TaskUpdate</code> patches one item by <code>taskId</code>
Match <code>block.name === "TodoWrite"</code>	Match <code>block.name === "TaskCreate"</code> or <code>"TaskUpdate"</code>
Item shape: <code>{ content, status, activeForm }</code>	<code>TaskCreate</code> input: <code>{ subject, description, activeForm?, metadata? }</code> . <code>TaskUpdate</code> input: <code>{ taskId, status?, subject?, description?, activeForm?, addBlocks?, addBlockedBy?, owner?, metadata? }</code> . <code>status</code> is <code>"pending"</code> , <code>"in_progress"</code> , or <code>"completed"</code> ; set <code>status: "deleted"</code> to delete
Render <code>block.input.todos</code> directly	Accumulate items across calls, or read a snapshot from a <code>TaskList</code> tool result

The assigned task ID is not in the `TaskCreate` input. It comes back in the matching `tool_result` as `{ task: { id, subject } }` , so capture it from the result block to key your map. The following example shows the minimal change to the Monitoring Todo Changes loop. To render a complete list, watch for a `TaskList` tool result in the stream or accumulate `TaskCreate` results and `TaskUpdate` inputs into a map.

The streamed `tool_use` input is the raw shape the model emitted. Claude Code repairs some close-but-incorrect key names before execution, mapping `id` or `task_id` to `taskId` and `active_form` to `activeForm` , but that repair is not reflected in the stream. Read `TaskUpdate`

input fields defensively, as the samples below do, rather than assuming the canonical name is always present.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Optimize my React app performance",
})) {
  if (message.type !== "assistant") continue;
  for (const block of message.message.content) {
    if (block.type !== "tool_use") continue;
    if (block.name === "TaskCreate") {
      const input = block.input as { subject: string };
      console.log(`+ ${input.subject}`);
    } else if (block.name === "TaskUpdate") {
      const input = block.input as {
        taskId?: string;
        id?: string;
        task_id?: string;
        status?: string;
      };
      const taskId = input.taskId ?? input.id ?? input.task_id;
      if (taskId && input.status) console.log(` ${taskId} ->
${input.status}`);
    }
  }
}
```

Related Documentation

- [TypeScript SDK Reference](#)
- [Python SDK Reference](#)
- [Streaming vs Single Mode](#)
- [Custom Tools](#)